

Kortix as a Backend

Wrap one shared agent as the backend for many of your end-users — connectors, model, secrets, and identity passed **by reference**. Your end-users never log in to Kortix.

The mental model

Like Stripe Connect or Twilio subaccounts: **Kortix authenticates your backend; your backend vouches for its end-user**. You hold one Kortix API key. Every session you start carries that user's connector profile, model, secrets, and an **origin_ref** attribution handle — all by reference, no per-user Kortix logins.

The end-to-end flow

- 1. Authenticate** Create an API key (Settings → Tokens). A **kortix_pat_...** token rides your project role, so sessions it starts auto-derive to **origin: backend** — on any plan, zero setup.
- 2. Mint a connector** Define a headless connector (provider `mcp/http/openapi/graphql`, static bearer — no OAuth), then mint a per-end-user **connection profile** with `owner_type: 'external'` (your app's user — the independent layer).
- 3. Start a backend session** Bind the profile via `connector_bindings`, pin the model/agent, vouch via `origin_ref`, and narrow secrets.
- 4. Stream the answer** `ensureReady() → stream() → send()` — live text deltas to your terminal or your own SSE endpoint.

The runnable wrapper

A single self-contained file does the whole flow:

`packages/sdk/examples/09-kaab-backend-wrapper.ts`. It typechecks against the SDK and runs on Bun.

One-shot (streams a turn to stdout)

```
KORTIX_API_URL=http://localhost:8008/v1 \  
KORTIX_API_KEY=kortix_pat_... KORTIX_PROJECT_ID=... \  
bun run examples/09-kaab-backend-wrapper.ts "Summarize my new signups"
```

Multi-tenant service (POST /run {endUserId, prompt} → SSE)

```
MODE=serve KORTIX_API_URL=... KORTIX_API_KEY=kortix_pat_... KORTIX_PROJECT_ID=... \  
bun run examples/09-kaab-backend-wrapper.ts
```

```
curl -N localhost:8791/run -H 'content-type: application/json' \  
-d '{"endUserId":"alice","prompt":"Summarize my new signups"}'
```

It uses `createScopedKortix` (one client per request, so concurrent end-users never race) and `narrowChatEvent` for the event stream. `KAAB_OVERRIDES=off` drops the backend-only fields so the connector + session + streaming path still runs against a deployment that doesn't have them yet.

The SDK calls, in order

```
import { createScopedKortix } from '@kortix/sdk/server';
import { narrowChatEvent } from '@kortix/sdk';

const kortix = createScopedKortix({ backendUrl, getToken: async () => API_KEY });
const project = kortix.project(PROJECT_ID);

// 1. connector definition (headless)
await project.connectors.create({ slug: 'user-mcp', provider: 'mcp',
  transport: 'http', url: MCP_URL, credential: 'shared',
  auth: { type: 'bearer', in: 'header', name: 'Authorization', prefix: 'Bearer ' } });

// 2. per-end-user profile (the independent, by-reference layer)
const prof = await project.connectors.profiles.reconcile({
  connector_alias: 'user-mcp', owner_type: 'external',
  owner_id: 'app-user-123', label: 'MCP for user 123' });
await project.connectors.profiles.updateCredential(prof.profile_id,
  { value: usersOwnToken, kind: 'secret' });
await project.connectors.profiles.activate(prof.profile_id);

// 3. backend-origin session
const s = await project.sessions.create({
  origin_ref: 'app-user-123',
  connector_bindings: { 'user-mcp': { profile_id: prof.profile_id } },
  opencode_model: 'anthropic/claude-opus-4-8', secrets: ['STRIPE_KEY'] });

// 4. stream the turn
const sess = kortix.session(PROJECT_ID, s.session_id);
await sess.ensureReady();
const h = await sess.stream({ onEvent: e => {
  const ev = narrowChatEvent(e);
  if (ev?.type === 'message.part.updated' && ev.part.type === 'text')
    process.stdout.write(ev.part.text);
  if (ev?.type === 'session.idle') h.close();
} });
await sess.send(prompt);
```

Verified live

Run against a local API on the KaaB branch, authenticated with a real kortix_pat_ token:

```
POST /sessions {origin_ref, opencode_model, agent_name}
-> 201 origin: backend origin_ref: tenant-42 agent_name: kortix
POST /sessions {secrets: ['GMAIL_TOKEN']}
-> 201 secrets_allowlist: ['GMAIL_TOKEN'] (narrowed)
POST /sessions {secrets: ['DOES_NOT_EXIST']}
-> 404 SECRET_IDENTIFIER_NOT_FOUND
POST /executor/.../connectors {slug, provider: mcp, url}
-> 200 ok: true (connector created)
wrapper (live) -> session origin=backend origin_ref=demo-user
connector not in manifest -> degrades to no binding (as designed)
ensureReady() now polls the cold start to ready (SDK fix)
```

ensureReady() polls the sandbox cold start until the runtime is ready. Streaming a real agent response then needs the sandbox to reach your API's KORTIX_URL — a hosted deployment works out of the box; for a LOCAL API front it with a public tunnel (cloudflared tunnel --url http://localhost:8010) since a cloud sandbox can't reach localhost. The streaming path is proven by the SDK tests + example 02.

Connectors: independent + backend-only

Two layers, both mintable with your API key, no browser:

- a) **The connector definition** (project-wide) — what/where it is. Headless for `mcp/http/openapi/graphql`. (pipedream needs interactive OAuth — not headless.)
- b) **A per-end-user connection profile** — the independent, by-reference layer. `owner_type`: `'external'` makes it belong to *your app's user*, not a Kortix member or agent, so it's usable purely by reference in a backend session. The broker resolves the credential **server-side at call time — it never enters the sandbox**.

All-or-nothing: if a session's `connector_bindings` sets any alias, every unbound alias resolves to null for that session — bind every connector the agent needs in one call. The connector must be declared in the project's `kortix.yaml` for per-user profiles to reconcile.

Session provenance — who/how it started

Field	Meaning
<code>origin</code>	Policy class: <code>user</code> <code>trigger</code> <code>schedule</code> <code>backend</code> <code>system</code> . Derived from the token kind, never the body. Your API key / PAT / service account → backend.
<code>origin_ref</code>	The end-user a backend session vouches for (→ <code>KORTIX_ORIGIN_REF</code> in the sandbox). Attribution only.
<code>secrets_allowlist</code>	The per-session secret narrowing that was applied.
<code>metadata.source</code>	The surface: <code>ui</code> <code>slack</code> <code>cli</code> <code>trigger:cron</code> ...
<code>created_by</code> / <code>owner_type</code>	Attribution: <code>user</code> <code>service_account</code> <code>unknown</code> .

Errors you may hit

Status / code	Meaning
403 <code>origin_override_forbidden</code>	A non-backend caller tried to set <code>origin_ref</code> or <code>secrets</code> . Use an API key / service-account bearer.
404 <code>SECRET_IDENTIFIER_NOT_FOUND</code>	An allowlisted secret identifier doesn't exist in the project.
409 <code>SECRET_IDENTIFIER_KEY_COLLISION</code>	Two allowlisted identifiers resolve to the same env var — name only one.
503 <code>KORTIX_URL_UNREACHABLE</code>	The sandbox can't call back (loopback <code>KORTIX_URL</code> / no tunnel). Run <code>pnpm dev</code> or set a public URL.
400 <code>INVALID_SESSION_SECRETS</code> / <code>_CONNECTOR_BINDINGS</code>	Malformed secrets / <code>connector_bindings</code> .

Environment

Var	Value
KORTIX_API_URL	Base incl. /v1 — http://localhost:8008/v1 (local) or https://api.kortix.com/v1 (prod)
KORTIX_API_KEY	Your kortix_pat_ token (Settings → Tokens → Create API key)
KORTIX_PROJECT_ID	The project the shared agent lives in
KAAB_OVERRIDES	off = drop backend-only fields (main-compatible); default on
KAAB_NO_CONNECTOR	1 = skip the connector layer (bare project)

Full reference: [docs/KORTIX_AS_A_BACKEND_GUIDE.md](#) · runnable example:
[packages/sdk/examples/09-kaab-backend-wrapper.ts](#) · SDK getting-started: [packages/sdk/GETTING-STARTED.md](#)